



University of
Southern Denmark

2nd Semester Project - HeatItOn

Heat Production Optimization

Group 8

Adela Pastekova – adpas25@student.sdu.dk

Alex Zalanyi – alzal25@student.sdu.dk

Filip Tichy – fitic25@student.sdu.dk

Marija Savkina – masav25@student.sdu.dk

Melana Ohrimeca – meohr25@student.sdu.dk

Zoltan Suveges – zosuv25@student.sdu.dk

May 28, 2026

Contents

Abstract	5
0.1 Abstract	5
1 Introduction	1
1.1 Introduction	1
1.2 Functional Requirements	4
1.3 Non-Functional Requirements	7
1.4 Target User Persona	8
1.5 System Requirements	8
2 Scrum Implementation	10
2.1 Scrum Implementation	10
3 Sprints	12
3.1 Sprints	12
4 Solution(Technical Aspect)	13
4.1 Verb-noun analysis	13
4.2 Classes and Methods Mapping	15
4.3 System Design	17
4.3.1 UML Diagram - Class Diagram	17
4.3.2 UML Diagram - Package Diagram	19
4.3.3 UML Diagram - Activity Diagram (Add Asset)	21
4.3.4 UML Diagram - Activity Diagram (Import Sources)	22
4.3.5 UML Diagram - Activity Diagram (Delete Asset)	23
4.3.6 UML Diagram - Activity Diagram (Optimize)	24
4.3.7 UML Diagram - Sequence Diagram (Optimize)	25
4.3.8 UML Diagram - Sequence Diagram (Refresh)	26
4.4 System Technical Architecture	27
5 Implementation and Technical Realization	29
5.1 Frontend Implementation	29
5.1.1 Midterm Version	30
5.1.2 Final Version	32

5.2	Backend Implementation	33
5.2.1	Midterm Version	33
5.2.2	Final Version	36
5.3	Data Management	38
5.3.1	Midterm Version	38
5.3.2	Final Version	39
6	Discussion and Reflection	41
6.1	Scrum	41
6.2	Requirements Engineering	42
6.3	Refactoring	42
6.4	Code review	43
6.5	Testing	43
7	Conclusion	44
7.1	Conclusion	44
8	References	46
8.1	References	46
9	AI Usage	47
A	Appendix	49
A.1	Appendices	49

List of Figures

1.1	System Architecture Diagram	2
4.1	HeatItOn Abstract Class UML Diagram	17
4.2	Package UML Diagram - Five main packages of the system. .	19
4.3	Add Asset Activity UML Diagram - Three parts and their actions for adding a new Asset.	21
4.4	Import Sources Activity UML Diagram - Three parts and their actions for importing Sources from a csv file.	22
4.5	Delete Asset Activity UML Diagram - Three parts and their actions for deleting Assets.	23
4.6	Optimization Activity UML Diagram - Three parts and their actions for deleting Assets.	24
4.7	Sequence UML diagram of optimization process	25
4.8	Sequence UML diagram of refreshing process	26
5.1	Assets tab from the demo version	31
5.2	Results tab from the demo version	31
5.3	Backend file structure showing the organization of controllers, services, interfaces, models, and data components.	34
5.4	Database schema illustrating the relationships between Source, OptimizedResults, ResultList, Result, and Asset tables. . . .	38
A.1	Full frontend and backend detailed UML class diagram. . . .	50

List of Tables

[

Abstract]Abstract

0.1 Abstract

Manually planning all the heat schedules might be time-consuming and inefficient. However, the heat production in the city of Heatington is currently managed that way, while supplying the heat to around 1600 buildings.

This project - HeatItOn, represents the development of a web application, providing an automated heat production platform. The project was developed using .NET C# and a MySQL database. API clients were used to provide communication between the backend and frontend.

The frontend was developed using Avalonia, providing a user-friendly interface with separate tabs for scenario management, source data input, and results visualization. The application consists of several modules: an Asset Manager responsible for managing assets;

a Source Data Manager responsible for processing incoming data; an Optimizer responsible for the main heat production optimization logic; a Result Data Manager responsible for processing output data; and a Data Visualization module responsible for presenting data through graphs and tables.

The system enables users to input source data, select assets for optimization, and analyze the generated results through tables and graphical visualizations.

Chapter 1

Introduction

1.1 Introduction

Problem Statement

HeatItOn is a utility in the city of Heatington that supplies heat to approximately 1600 buildings. HeatItOn currently plans heat production manually, which makes it hard to always choose the cheapest combination of production units. The problem is therefore to develop an optimization system that can automatically create heat production schedules that always meet the heat demand while minimizing total costs and maximizing profit from electricity production/consumption.

Problem Analysis

HeatItOn is a utility in the city of Heatington that supplies heat to approximately 1600 buildings through a district heating network. Heat is produced by traditional heat-only boilers and by units that both produce heat and produce or consume electricity (CHPs). Most of the heat comes from cogeneration plants that use different fuels such as waste, gas, coal, and biomass.

District heating works by pumping hot water from a plant through insulated pipes to buildings, where it is used for space heating and hot tap water, then cooled water then flows back to the plant to be reheated.

Today, HeatItOn plans its heat production manually. This makes it difficult to choose the best combination of units and to react to changing electricity prices in a systematic way. Our task is to develop software that optimizes heat production and creates heat schedules for all available units with the lowest possible costs and the highest possible profit from the electricity market.

System will consist of 5 main modules:

- Optimizer (OPT)
- Data Visualization (DV)
- Asset Manager (AM)
- Source Data Manager (SDM)
- Result Data Manager (RDM)

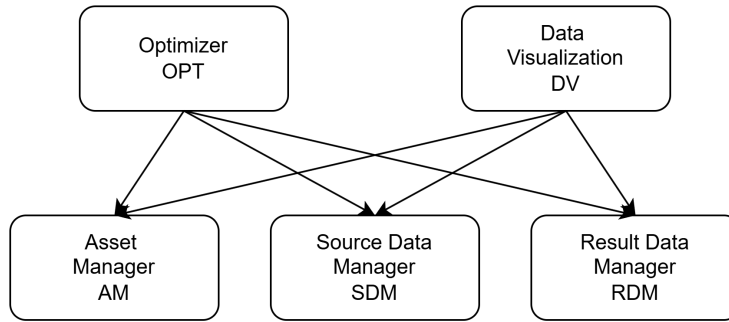


Figure 1.1: System Architecture Diagram

Optimizer (OPT):

The OPT considers all the available production units and calculates the best economical schedule of the heat production while securing heat availability to all buildings at any time. All units have a specific production cost, expressed in DKK per MWh produced heat. The source of this information is excel file called 2026 Heat Production Optimization - Danfoss Deliveries

Data Visualization (DV):

The DV is responsible for a graphical presentation of important source and result data.

Asset Manager (AM):

The AM is a repository for static system information and is responsible for making this information available for other modules. Through the AM, other modules can retrieve heating grid information such as name and a graphical representation of the grid.

Source Data Manager (SDM):

The SDM is a repository for dynamic system information. The SDM can hold its data in local CSV files. The SDM provides time series for the heat demand and electricity prices. Both the heat demand and the electricity prices are given with a resolution of 1 hour.

Result Data Manager (RDM):

The RDM is a repository for optimization results

Scenarios

Scenario 1 (Heat only)

Scenario 1 uses a single district heating network with three gas boilers (GB1, GB2, GB3) and one oil boiler (OB1). The gas boilers are cheaper, while the oil boiler has higher production costs and is mainly used in peak demand periods. All boilers have constant production costs that do not depend on the hour of operation. Strategy for scenario one is to run the cheapest gas boiler first and then add production from the more expensive boilers if needed to meet the heat demand. There should also be chosen period of 30 to 60 hours when one of the gas boiler will be down due to maintenance.

Scenario 2 (Heat and electricity)

Scenario 2 uses two gas boilers (GB1, GB3), one gas motor (GM1), and one electric boiler (EB1). The gas motor produces electricity (which can be sold), while the electric boiler consumes electricity (which must be bought). When electricity prices are high gas motor has to be running because selling electricity generates extra income. When electricity prices are low electric boiler has to be running because electricity is cheap to buy.

To decide which units to use in each hour, net production cost will be calculated for each unit and then compared in order to figure out best result.

This project will be using data delivered by Danfoss, including:

- Graphical representation of the district heating network
- Configuration parameters for each production unit (image, produced heat, produced/consumed electricity, primary energy consumption, production costs, CO₂ emissions)
- Heat demand and electricity prices for a 14-day winter period with 1-hour resolution
- Heat demand and electricity prices for a 14-day summer period with 1-hour resolution

If there is time left, project can be extended by usage of other visualization libraries, storing data in a database, enabling or disabling units directly from the UI, running alternative configurations in Scenario 2, and implementing APIs and an API client for reading electricity prices from Energinet.dk.

1.2 Functional Requirements

Scenario 1

1. Source Data Manager (SDM) & Result Data Manager (RDM)

Components that handle where the data comes from and where it goes.

- The user should be able to receive input data.
- The user should be able to retrieve input data.
- The user should be able to output data. [**Database integration**]

2. Asset Manager (AM)

Component that keeps track of the physical boilers and their constraints.

- The user should be able to upload assets from csv file.
- The user should be able to add asset.
- The user should be able to edit assets. [**Boiler definitions**]
- The user should restrict the maximum heat output of each unit as follows: GB1 to 3.0 MW, GB2 to 2.0 MW, GB3 to 4.0 MW, and OB1 to 6.0 MW. [**Capacity constraints**]

3. Optimizer (OPT)

- The user should be able to schedule the cheapest boilers first to fulfill the district's heat demand. [**Cost-based dispatch**]
- Based on production costs, the optimizer should prioritize boilers in the following strict order: GB1 (510 DKK), GB2 (540 DKK), GB3 (580 DKK), OB1 (690 DKK). [**Dispatch order**]
- The user should only use the more expensive oil boiler (OB1) to supplement heat during peak demand periods when the gas boilers cannot fulfil the total demand. [**Peak demand handling**]
- The user should be capable of optimizing units separately for both a winter period and a summer period. [**Seasonal optimization**]
- The optimizer should remove a designated gas boiler from the available pool of units during its scheduled maintenance period, ensuring it produces 0 MW. [**Maintenance logic**]

4. Data Visualization (DV) / User Interface

- The UI should validate that the user's selected maintenance duration is a minimum of 30 hours and a maximum of 60 hours. [**Maintenance constraints validation**]
- The UI should display a chart or graph that visually compares the total heat demand against the individual heat output contributions of GB1, GB2, GB3, and OB1 over time. [**Result graphing**]
- The UI should provide a way for the user to choose an obligatory maintenance period for one unit in both the summer and winter periods. [**Maintenance scheduling**]

Scenario 2

1. Source Data Manager (SDM) & Result Data Manager (RDM)

Component that handle a crucial data stream: electricity prices.

- The user should be able to retrieve hourly electricity prices data from data base.
- The user should be able to retrieve hourly heat demand data from the database. [**Market data retrieval**]

2. Asset Manager (AM)

The physical assets include units that interact with the power grid.

- The user should be able to upload assets from csv file. [**Unit registration**]
- The user should be able to limit the maximum heat output of each unit: GB1 to 3.0 MW, GB3 to 4.0 MW, GM1 to 5.3 MW, and EB1 to 6.0 MW. [**Heat capacity constraints**]
- The user should account for electricity production and consumption capacities, specifically noting that GM1 produces up to 3.9 MW of electricity, while EB1 consumes up to 6.0 MW (-6.0 MW). [**Electricity constraints**]

3. Optimizer (OPT)

- The user should be able to calculate the net production costs for the electricity-producing/consuming units based on the fluctuating electricity prices. [**Dynamic net cost calculation**]

- The optimizer should prioritize running the gas motor (GM1) during periods of high electricity prices. [**High-price optimization logic**]
- The optimizer should prioritize running the electric boiler (EB1) during periods of low electricity prices. [**Low-price optimization logic**]
- The optimizer should actively prevent the gas motor (GM1) from running during summer periods with negative electricity prices (to avoid paying for produced electricity).
- The optimizer should recognize the electric boiler as an extra income source during negative price hours. [**Negative price handling**]
- The optimizer should completely remove either GB1 or GM1 from the optimization calculations during its scheduled maintenance period. [**Maintenance logic**]

4. Data Visualization (DV) / User Interface

- The UI should allow the user to select an obligatory maintenance period for either GB1 or GM1, enforcing a duration between 30 and 60 hours. [**Maintenance scheduling**]
- The UI should display a graph comparing the total heat demand against the individual heat output contributions of GB1, GB3, GM1, and EB1 over time. [**Result graphing**]

General App Navigation & UI

Requirements that cover the flow of the application from the user's perspective.

- Upon launch, the application should display a home screen allowing the user to explicitly select between Scenario 1 and Scenario 2. [**Startup Page**]
- The user interface should allow user to navigate between scenarios easily. [**Scenario navigation**]

System Architecture & Data Handling

Requirements that cover how the app manages data and how the 5 components talk to each other.

- The user should be able to use a database system to save and retrieve data. [**Database storage**]

Advanced Configuration & Comparison

Requirements that give the user more control over the optimization process.

- The UI should provide a mechanism for the user to dynamically activate or deactivate individual heating units from being considered in the optimizer's calculations. [**Dynamic unit toggling**]
- The system should allow the user to run custom scenario with configurations setted by user. [**Custom scenario configuration**]
- The Data Visualization module should provide results of different configured scenarios for result comparison. [**Comparison metrics**]

1.3 Non-Functional Requirements

1. Performance & Efficiency

- The system should complete all optimization calculations within 3 seconds of the user clicking or "Optimize." [**Optimization speed**]
- The system should completely render the resulting graphs within 3 seconds of the user clicking "Run". [**Processing speed**]
- The system and database should be capable of efficiently loading, processing, and aggregating a full month of hourly data (approx. 720-744 data points) without UI freezing or lag. [**Data volume handling**]

2. Environment & Portability

- The application should be fully cross-platform compatible, executing reliably on Windows, macOS, and Linux operating systems. [**Cross-platform compatibility**]

3. Usability & Localization

- The entire user interface, including all menus, labels, and error messages, should be written in English. [**UI Language**]
- All financial values, production costs, and revenues should be calculated and displayed strictly using Danish Kroner (DKK). [**Currency localization**]
- The resulting visualization for a month of data should be highly readable, formatting the daily boiler usage as a stacked chart (matching the total heat demand on the Y-axis, and days of the month on the X-axis) alongside a daily cost graph. [**Visualization clarity**]

4. Reliability & Robustness

- The system should not crash if the Energinet.dk API is unresponsive or if the user loses internet connection. [**System stability**]
- In the case of an API failure, the system should handle the error by displaying a clear, understandable error feedback message or popup to the user. [**Exception handling**]

1.4 Target User Persona

The system is designed with a specific target audience in mind. To ensure an optimal user experience, the ideal user (persona) interacting with this system is assumed to have the following profile:

- Be able to read in English
- Be able to navigate a simple UI
- Have a basic understanding of the district heating problem being solved by the system
- Be able to read and understand basic input/output data

1.5 System Requirements

The system is split into three parts: Front End, Back End and the Database. Both the Backend and the Database are run in a Linux Docker container. The Frontend is run locally on the user's machine. For a machine to run the project they need the following:

Software Specifications

Entire project: .NET 9.0

Backend: Docker Desktop (latest version)

Database: Docker Desktop with MySql Image (shall be installed automatically if missing)

Frontend: Required NuGet packages (all must be compatible with .NET 9.0):

- Avalonia.UI

Hardware Requirements

- WSL version 2.1.5 or later (for Windows)
- Any of the following OS's: Windows 10/11 (64 bit), MacOS, Linux (64 bit)
- 4GB of RAM
- Motherboard supporting hardware optimization
- 3GB of free space

Chapter 2

Scrum Implementation

2.1 Scrum Implementation

This chapter presents how the team implemented Scrum and its ceremonies from the very first week of development. Development consisted of six two-week iterative sprints. The team started with sprint planning, where they created a backlog of tasks and assigned story points to them. During each sprint, team members worked on their assigned tasks and held regular standup meetings to discuss progress and any obstacles they encountered. At the end of each sprint, the team held a sprint review to demonstrate their work and gather feedback, followed by a sprint retrospective to reflect on the process and identify areas for improvement. This iterative approach allowed the team to continuously improve its workflow and deliver a high-quality product by the end of the project.

The Scrum master was elected at the beginning, and because the member representing this role was highly skilled and the rest of the team agreed, he remained in this role until the end of the project. All team members agreed to use Jira. Jira was chosen because it allowed the team to use all features, including task management and proper story point assignment, without requiring additional subscriptions. Team members had extensive experience with Jira, as it allowed them to understand the complexity of the project more easily. The backlog was used to store future tasks, and certain tasks were transferred from the backlog to the sprint backlog during sprint planning. The board for each sprint consisted of several columns: “TO DO”, “IN PROGRESS”, “UNIT TESTING”, “UNDER REVIEW”, and “DONE”. Furthermore, one member integrated Jira with the IDEs, so when a team member started working on a new branch, the task automatically moved to “IN PROGRESS”. A crucial aspect of team cooperation was the approach to code reviews. Every task required a minimum of two reviewers. If a reviewer identified any issues or deficiencies, a change request was created in GitHub, and the branch owner had to resolve it before the

reviewer would grant approval again.

The project was completed over six sprints. Each sprint consisted of the necessary Scrum ceremonies. Based on the project requirements, the team scheduled two standup meetings per week, specifically on Mondays and Wednesdays. These standup meetings remained consistent throughout the semester, serving as a short and structured way to synchronize progress, address problems, and ensure effective communication until the end of the project.

Sprint planning was held every other week before the start of a new sprint. Each planning session was scheduled for four hours. For the first two sprints, planning took place on Mondays. After this period, the team decided to move sprint planning to Friday of the preceding week. Monday was a particularly busy day for all team members, and moving planning to Friday allowed the team to use the weekend for preparation, which made the transition between sprints smoother. With each subsequent sprint planning, less time was required as team members got better at planning, focusing, and deciding which tasks needed to be done and who was the right person for the job.

At the end of each sprint, the team held two final ceremonies: the sprint review and the sprint retrospective. Both ceremonies were held on Fridays.

Initially, all deadlines for tasks and pull-request reviews were set for Thursday evening. However, the team found this approach ineffective, and there was not enough time for reviews to be completed before the end of the sprint. The deadline for tasks was later moved to Wednesday evening, and the deadline for reviews to Thursday evening. This adjustment ensured there was enough time for all ceremonies held on Friday.

During the sprint review, team members presented what each of them had accomplished over the past two weeks. This session, along with the retrospective, was especially important for the team. It provided valuable insights into each member's progress, task preferences, and skill levels, allowing everyone to understand how the work was executed. The sprint retrospective was conducted through Jira. For each sprint, one team member established a table with three columns: "Start doing", "Stop doing", and "Keep doing". The team was given ten minutes to submit anonymous feedback, after which everyone reviewed and discussed the compiled topics. This ceremony turned out to be just as helpful as the sprint review. The retrospective was the ceremony that contributed the most to the team's growth and ability to cooperate effectively.

Chapter 3

Sprints

3.1 Sprints

This is our Sprints.

Chapter 4

Solution(Technical Aspect)

This section will outline the process of creation of the project, from project description to final solution, also describing the technical architecture. The solution began with going through the provided Project Description. From here important parts were selected and wrote into a compact version. This compact version was suitable for conducting the verb/noun analysis where class and method candidates emerged. The class candidates are able to be transformed into CRC cards outlining the responsibilities and collaborators of each class. The CRC cards are used to create UML diagrams(Class,Package,Activity and Sequence). The different diagrams later on were used for implementation.

4.1 Verb-noun analysis

Asset Manager (AM) should store its data in local files and supply other modules with it. Source Data Manager (SDM) should store dynamic data in csv files. Result Data Manager (RDM) should save result data in local CSV files. Optimizer (OPT) should calculate the best economical schedule of the heat production for a given period. It also secures heat availability to all buildings at any time. There are different units, each has a specific production cost, expressed in DKK per MWh produced heat. For heat only units there are no further expenses nor income. Electricity producing units should sell the produced electricity and have thereby an extra income depending on the electricity prices which can change from hour to hour. Electricity consuming units should buy the electricity and have thereby an extra expense, again depending on the electricity prices. Net production cost should be calculated differently for each unit type and compared to build priorities. The Data Visualization should display a graphical presentation of important source and result data.

Candidates for classes:

- | | | |
|-----------------------|---------------------|-----------------------|
| • Asset Manager | • Schedule | • Expenses |
| • Data | • Production | • Prices |
| • Files | • Period | • Net production cost |
| • Modules | • Heat availability | • Unit type |
| • Source Data Manager | • Buildings | • Data Visualization |
| • Result Data Manager | • Time | • Presentation |
| | • Electricity | • Priorities |
| • Optimizer | • Units (GB1-EB1) | |
| | • Income | |

Candidates for methods:

- | | | |
|-------------|-------------|-----------|
| • Store | • Has | • Buy |
| • Supply | • Sell | |
| • Save | • Have | • Compare |
| • Calculate | • Depending | |
| • Secures | • Change | • Display |

4.2 Classes and Methods Mapping

Class Name	Associated Methods
AssetManager	store, supply
SourceDataManager	store (dynamic data), read CSV
ResultDataManager	save (results)
Optimizer	calculate (schedule), secure (heat availability), compare
Unit	sell (electricity), buy (electricity), calculate net production cost
DataVisualization	display (graphical data)

CRC Cards

Class: Asset Manager	
Responsibilities	Collaborators
Store heater data	- Optimizer - Data Visualization - Unit

Class: Optimizer	
Responsibilities	Collaborators
Calculates heating schedule	- Asset Manager - Source Data Manager - Result Data Manager

Class: Source Data Manager	
Responsibilities	Collaborators
Stores market data	- Optimizer - Data Visualization

Class: Result Data Manager	
Responsibilities	Collaborators
• Save output data	• Data Visualization • Optimizer

Class: Data Visualization	
Responsibilities	Collaborators
• Display important data to the user	• Source Data Manager • Result Data Manager • Asset Manager

Class: Units	
Responsibilities	Collaborators
• Contains data for different heaters • Calculate net production costs	• Optimizer • Asset Manager

4.3 System Design

4.3.1 UML Diagram - Class Diagram

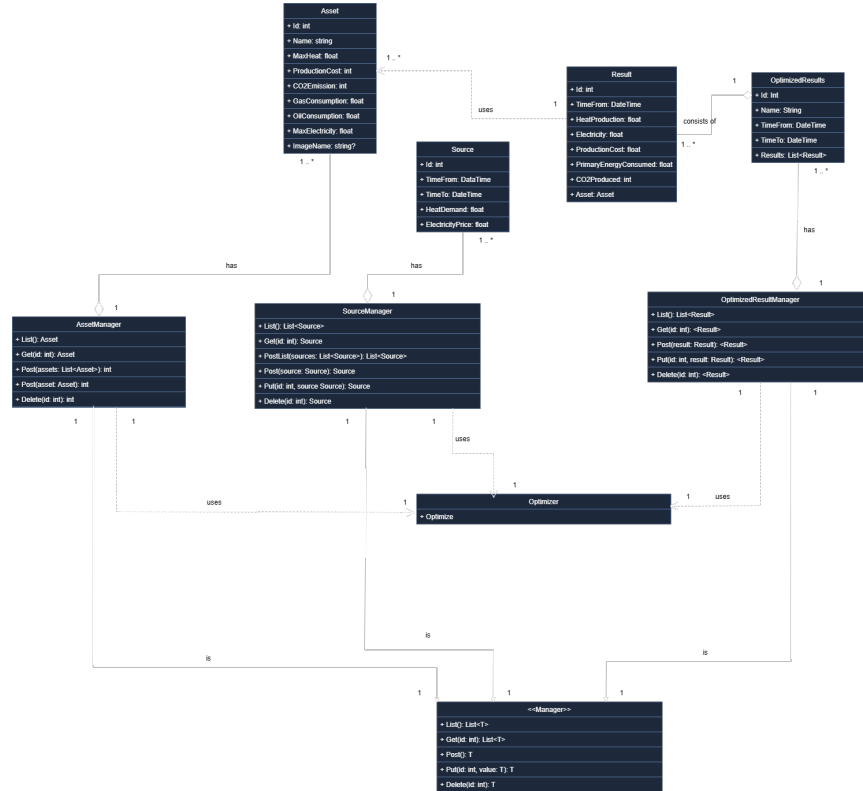


Figure 4.1: HeatItOn Abstract Class UML Diagram

The diagram (Figure 4.1) represents the unified system architecture. This is a preliminary version that integrates both the UI components (Frontend) and the logic/data services (Backend).

4.3.2 UML Diagram - Package Diagram

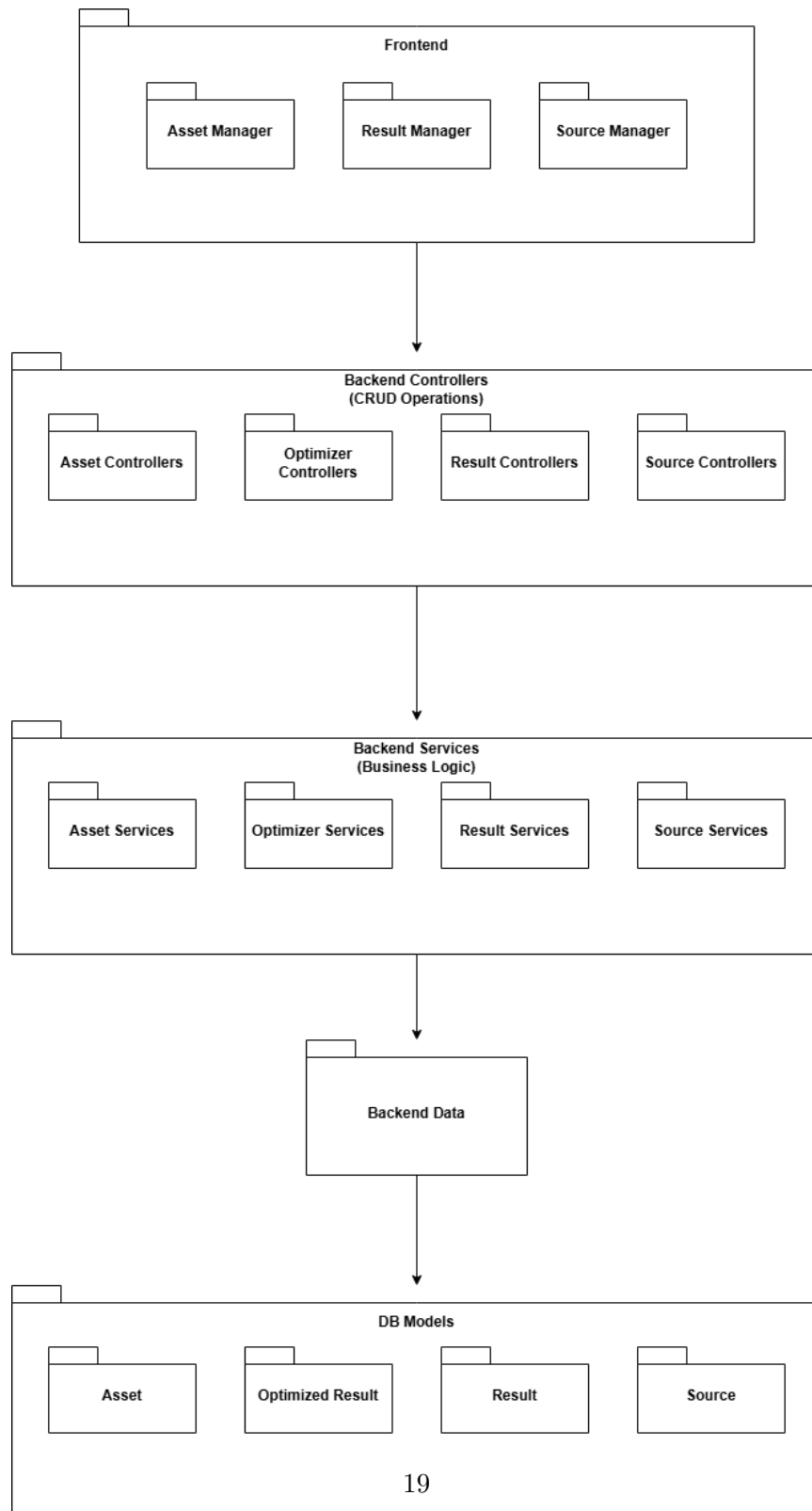


Figure 4.2: Package UML Diagram - Five main packages of the system.

The package diagram shown (Figure A.1) simplified the system into five packages.

The first one is the Frontend package, containing the Asset, Result and Source Managers. This is the UI of the system, with what the user will interact. The Frontend is following the Model-View-ViewModel pattern, layering the different parts of each manager. This package connects with the next package through clients.

The second package, the Backend Controllers, contain the CRUD(Create, Read, Update, Delete) operations of each module. Through HTTP requests these can be accessed and will call the services in the next package.

Backend Services are the third package containing all the business logic (like the Optimization algorithm).

The services communicate with the Database for which the connection, the fourth package, is located inside the Data layer. The Database is responsible for storing elements of the fifth package (Asset, Source, Result), the models.

This diagram shows a very simplified relationship with the most important parts of the overall system, giving an understanding of the overall workings and layers. Although it is very simple, it is a good starting point for understanding the entire architecture.

These packages, in the final solution, while expanded upon greatly, are the backbone of the implementation.

4.3.3 UML Diagram - Activity Diagram (Add Asset)

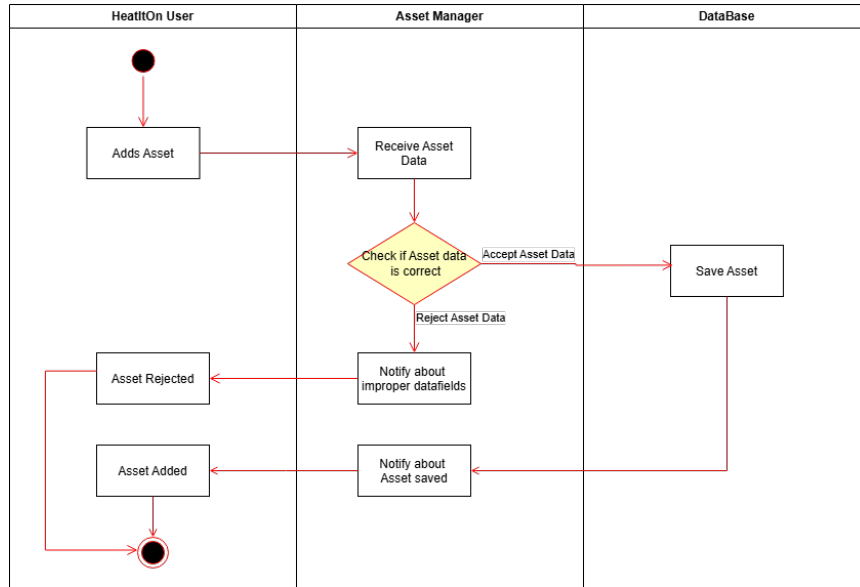


Figure 4.3: Add Asset Activity UML Diagram - Three parts and their actions for adding a new Asset.

The activity UML diagram (Figure 4.3) depicts the addition of a new Asset by the user into the system.

The activity is initiated by the user when they want to add a new Asset interacting with the Asset Manager. The filled out data is then verified by the Asset Manager. If the data was filled out correctly (like having a proper name or number fields having numbers) the Asset Manager will create the Asset and send it to the Database, who will save it. If the data was not filled out correctly, it will reject the received data. The activity ends with the user being notified of both cases, ending it. For a successful addition the Asset Manager will also display the new Asset instantly.

Adding an Asset is started in the Frontend. The Frontend clients will call the Backend that will handle the communication with the Database where the Asset will be saved. The Frontend deals with user input and handling the data as well.

This diagram helps getting an idea about how the communication, for adding Assets, works. This activity is crucial to the solution, as every Asset needs to be stored in the Database for the optimization to work.

4.3.4 UML Diagram - Activity Diagram (Import Sources)

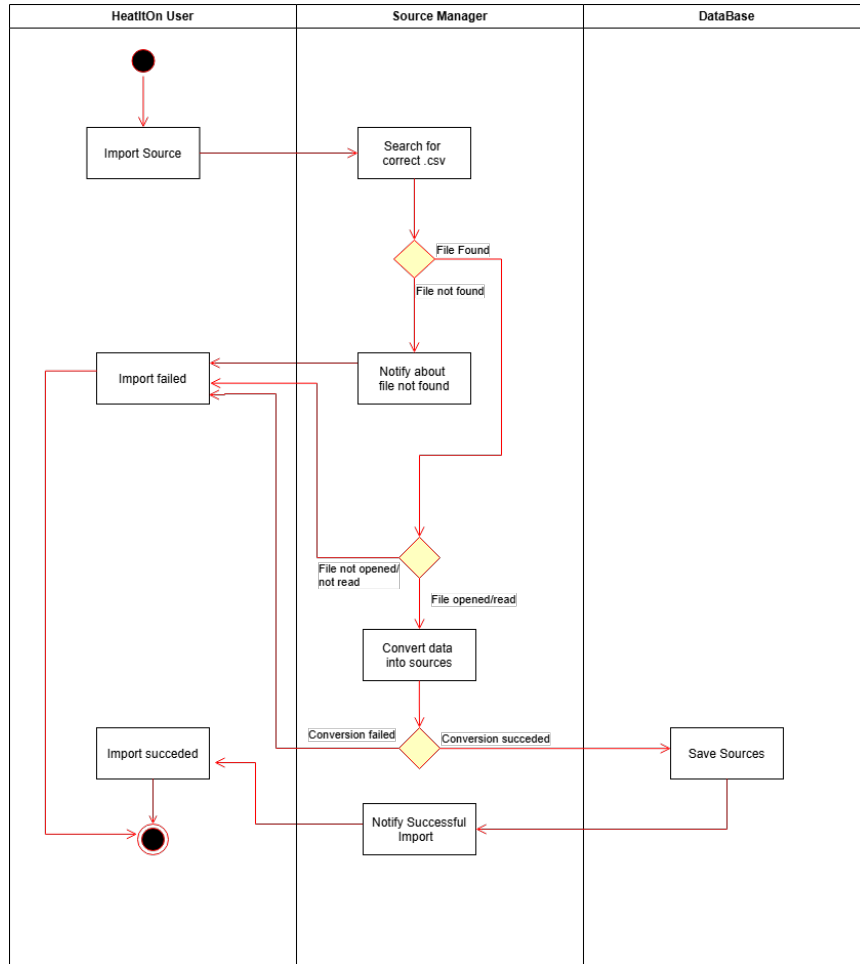


Figure 4.4: Import Sources Activity UML Diagram - Three parts and their actions for importing Sources from a csv file.

The activity UML diagram (Figure 4.4) depicts the importing of one or multiple Sources stored in a Comma Separated Value(csv) file.

Importing is initiated by the user interacting with the Source Manager UI. The Source Manager then will try to find the requested csv file. If the file is found the Source Manager will try to open and read the data in the Frontend. After a successful opening and reading the data will be transformed into proper Source models and will be sent over through the Backend to the Database for storage, notifying the user. If the file is not found or if the file cannot be opened or read properly, like if the csv is not structured properly, the importing process fails and the user is notified. The activity is concluded by notifying the user of either a successful or failed

import, the former is visible by a new data set appearing in the Source Manager.

This helps getting an understanding of how importing works for sources (but not only), and the workings of the Source Manager.

The Source Manager and Sources are the other crucial part of the system, without which, the optimization process could not be run.

4.3.5 UML Diagram - Activity Diagram (Delete Asset)

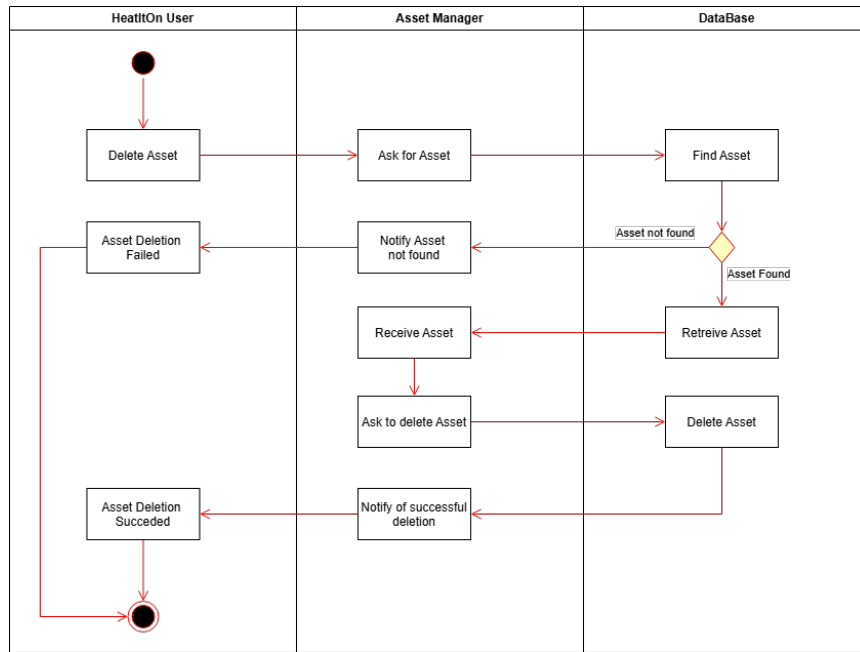


Figure 4.5: Delete Asset Activity UML Diagram - Three parts and their actions for deleting Assets.

The activity UML diagram (Figure 4.5) depicts the deletion of an Asset through the Asset Manager.

Deletion is initiated by the user through interacting with the Asset Manager UI. The Asset Manager will try to retrieve the Asset from the DataBase(through the Backend). If the DataBase finds the Asset it will return it to the Asset Manager. If the DataBase could not find the requested Asset, it will prompt the Asset Manager to notify the user ending the activity. If the Asset is found it will be retrieved to the Asset Manager who will request its deletion from the DataBase. The DataBase deletes the Asset and the Manager will notify the user of the activity being successful, after which it ends.

This activity diagram portrays another CRUD operation, in this case Deletion, in the system.

This helps getting an understanding of how deletion of objects works in the system, using Assets as an example. Deletion is crucial for the proper workings of the system.

Deletion is used with Assets, as seen on the diagram, but also with Results, where the user has the option to delete either of them.

4.3.6 UML Diagram - Activity Diagram (Optimize)

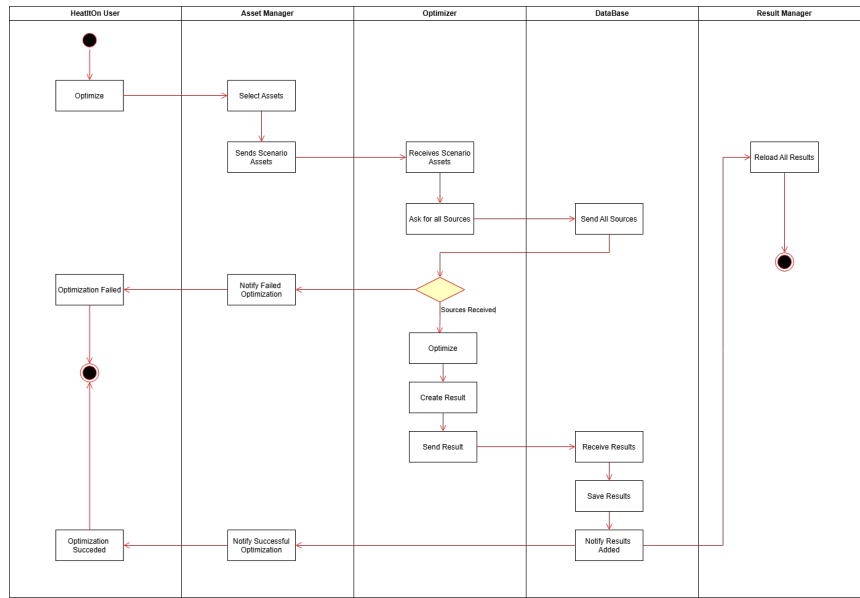


Figure 4.6: Optimization Activity UML Diagram - Three parts and their actions for deleting Assets.

The activity UML diagram (Figure 4.6) depicts the optimization process.

First, the optimization starts with the user, using the Asset Manager UI, selecting Assets and starting the optimization process. The Asset Manager will send the selected Assets to the Optimization module, which after receiving them, will call from the Database all stored Sources up to that moment. If no Source is received it will prompt the Asset Manager to notify the user and end the activity. The Optimizer has no UI elements and is completely in the Backend. If it receives Sources, it will run the optimization algorithm, which will create a Result. The new Result is sent and stored in the Database that will notify both the Asset Manager(that in turn notifies the user) and the Result Manager that there is a new Result ready to be displayed, ending the activity.

This activity diagram proves its relevancy by demonstrating the optimization activity, which is the entire purpose of the system. While it is not enough for a full understanding of the optimization process, the diagram illustrates the different overall steps happening.

Optimization is the part that interacts with every module of the system. It uses directly or indirectly every implemented Manager, and all the activity diagrams (and all their depicted actions) are just helping the optimization process depicted here.

4.3.7 UML Diagram - Sequence Diagram (Optimize)

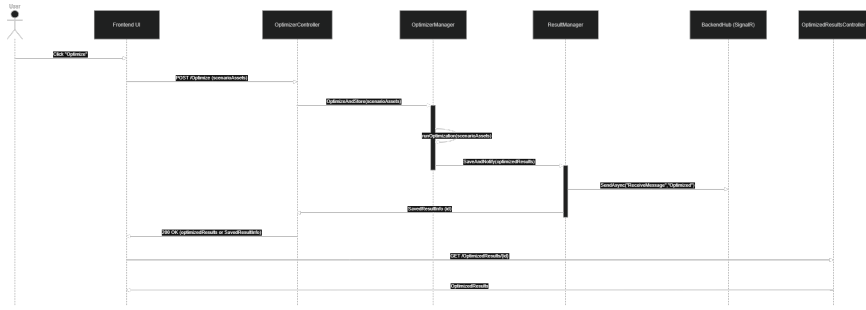


Figure 4.7: Sequence UML diagram of optimization process

The sequence UML diagram (Figure 4.7) depicts the optimization process.

The participants are: User, Frontend UI, ‘OptimizerController’, ‘OptimizerManager’ (runs the algorithm), ‘ResultManager’ (saves results and sends notifications), ‘BackendHub’ (SignalR) and ‘OptimizedResultsController’ (for later retrieval). The flow is: the user clicks Optimize in the frontend, the frontend sends the selected assets to ‘OptimizerController’ with ‘POST /Optimize’, ‘OptimizerController’ calls ‘OptimizerManager’ to run the optimization, ‘OptimizerManager’ produces an ‘OptimizedResults’ object and gives it to ‘ResultManager’ to save, ‘ResultManager’ writes the result to the database and asynchronously notifies clients via ‘BackendHub.SendAsync(“ReceiveMessage”, “Optimized”)’, the controller returns either the full result or a small confirmation (‘SavedResultInfo’ with the new id) to the frontend, the frontend can then request the stored result with ‘GET /OptimizedResults/id’ from ‘OptimizedResultsController’ to display the full persisted data. If the optimizer cannot find required source data, it returns an error/notification back to the frontend and the activity ends with the user being informed. In the diagram the ‘OptimizerManager’ and ‘ResultManager’ are shown as manager boxes that group the backend logic (they represent the real services and saving/notification steps) so the sequence stays easy to read while matching the actual system behavior.

This sequence diagram shows the main flow: how a user action becomes an optimized schedule and is delivered to the UI, connecting the user, back-end computation, storage and real-time updates so the reader sees the end-to-end behavior without needing the algorithm details.

How this maps to the code:

- The optimizer endpoint is in `OptimizerController`, the `OptimizerController` receives the `POST /Optimize` request.
- The optimization work runs in `OptimizerService` (this is the diagram's `OptimizerManager`).
- Saving the results is done by `OptimizedResultsService` (the diagram's `ResultManager`).
- Real-time notifications go through SignalR in `BackendHub`, controllers use `IHubContext<BackendHub>.SendAsync` to send messages.
- Fetching stored results is handled by `OptimizedResultsController`.
- The database model and migrations are in `BackendDbContext`.
- Centralized error handling (turning exceptions into HTTP responses) is implemented in `ExceptionHandlerMiddleware`.

4.3.8 UML Diagram - Sequence Diagram (Refresh)

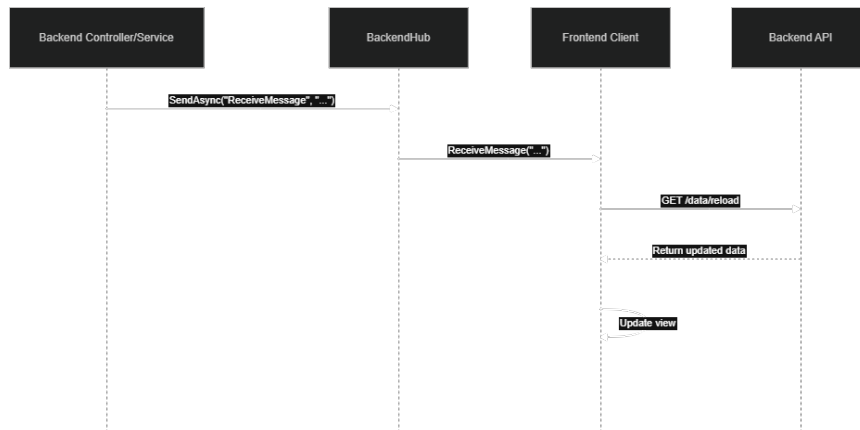


Figure 4.8: Sequence UML diagram of refreshing process

The refresh sequence diagram is relevant because it shows how the frontend stays synchronized with changes made in the backend, which is an important part of the system's behavior. It demonstrates the full update flow after a backend action has changed some data: the backend first sends

a SignalR notification and then the frontend reacts by loading the latest data again. This makes the diagram useful because it explains how the user sees updated information without manually reloading the page and it also shows the difference between pushing a notification and pulling the new data. The diagram connects directly to the implemented solution in the codebase: controllers and services use `IHubContext<BackendHub>` to call `SendAsync(...)`, the frontend receives the `ReceiveMessage` event and then it requests the updated data from the backend through endpoints such as `OptimizedResultsController` or another reload request. In this way, the diagram reflects the real implementation and shows how real-time updates are handled in the system.

4.4 System Technical Architecture

For the system architecture, from a technical standpoint, C# was chosen as the base technology stack. This was chosen as its .NET 9.0 version is suitable for the development of this system with its simplicity, the familiarity of the team and its object-oriented imperative style. Every other piece of technology is related to .NET and was chosen based on their respective compatibility with the language.

Technology Stack

The first chosen piece was the Database that is running MySQL. The role of the Database is to store the system data independently from other parts of the system (Backend and Frontend). It stores relevant user data, using models, like the Assets, Sources, Results of each generator and Results of optimization runs. The Database communicates with the Backend using Microsoft Entity Framework. The Frontend cannot communicate directly with the Database, it must do it through the Backend. The Database allows data storage that is independent from runs. The system can work with multiple users at the same time or with one user in different runs, storing all the data in the Database, where it will remain until deletion.

The second chosen piece of technology stack is for the Frontend UI. For this AvaloniaUI was chosen for its good compatibility with .NET and modularity. Besides this, the Frontend is following the Model-View-ViewModel (MVVM) design pattern.

Layering

The system is divided into three main parts, Frontend, Backend and the Database. Each serves a different role and, besides the Database, they are further separated for proper readability and modularity.

The Frontend, as stated earlier, follows the MVVM pattern. The Models, together with the Data (containing file I/O and clients) form the overall data and business layer. The View forms the presentation layer, containing all the UI elements. The ViewModel creates a connection layer between the two other parts.

The Backend is following a .NET web application template. Its role is to create a connection between the Frontend and Database with an Application Programming Interface (API), and to handle overall business logic for the entire system. For modularity, this is achieved through serious layering. The Backend consists of six main parts. Controllers handle incoming/outgoing traffic to/from the Database and Frontend and will call the appropriate Services. Services contain all the business logic of the system (like optimization or add Asset). Both Controllers and Services inherit from two Interfaces, as they share similarities (like CRUD operations). The Data layer is composed of two other parts, Models (storing data models) and Data (storing DB Migrations and DB Context which is the main connection point to the DB). The presentation layer is completed by the last element, Hubs, which ensures real time communication with the Frontend application (instantly notifies to refresh the Frontend). Although the Backend is divided into many pieces, this was necessary for modularity, readability and because of complexity as well. This way each small section is responsible of one part, making replacement easier. In this solution, if one wants to hypothetically change the entire optimization algorithm, they only need to rework the Optimization Service, without having to touch any other parts.

Architectural Style

The project follows all major design principles. The main set of principles that the project adheres to, is SOLID. This happens in both Frontend and Backend and is visible in many parts, like interfaces or being able to add new features without having to change code. SOLID principles help create an Object-Oriented design. But besides these, the project follows DRY (Do not Repeat Yourself), KISS (Keep It Simple, Stupid), YAGNI (You Aren't Gonna Need It) and SoC (Separation of Concerns).

The technology stack, the layering and the architectural style used are all necessary for the proper running of the system. While the technology stack was kept simple by only using what was necessary for achieving requirements, the layering and style help build a good quality project that is easy to understand, change and complete. But extensive layering also helps following the style, by breaking up the large task into small sections creating a *Divide et impera* style of development, making it easier to adhere to the principles.

Chapter 5

Implementation and Technical Realization

HeatItOn is divided into three sections, and two versions. First there is the midterm evaluation demo version, and then there is the final version. We will go through both to see the first implementation and the improvements and changes that we made for the final version.

5.1 Frontend Implementation

Our Frontend Implementation is designed with the plant operators in mind, keeping it simple and user friendly. To achieve this, we started with a UI-sketch where we tried to come up with an easy to use and easy to understand concept, that we tried to achieve in our implementation.

The application is split into three different tabs. The first one, that the user sees when they launch the application is the Assets tab. Here the user can manage and see all the existing assets, they can import or export assets as csv's or manually add/remove or edit specific assets one by one. This page also has three preset toggles(Scenario 1, Scenario 2 and Custom). As the Assets page is where the operator will start the optimization process, by selecting the asset combination they want to optimize with, the toggles merely select from the assets the ones that are in that scenario. If they wish to optimize for a new scenario, they can select the Custom toggle, allowing them to run the optimization process for any scenario. After selecting the desired assets the user has to click on the Optimize button that will start the optimization process for the selected assets.

The list of assets is connected through the backend with the database, each modification being done live in the database. The optimization process from the backend is called from the frontend by clicking a button. The Assets tab will send the Optimization algorithm the assets selected to be optimized every time the button is clicked.

The second tab, the Results tab is where, once the user started an optimization, the result of that run will appear. On the left side, they can see and select out of a list of all results. When they select one result, the user can see the hourly result in a table format or view the data in different graphs. They can also export them into csv's. If the user would like to focus on a specific time period, not the entire optimization run, they can use a date-time picker to select from when to until when they want to see the result data, which will affect the result table and the graphs as well.

The tab loads all the optimization-results from the database using the backend services.

The third tab, Sources, is where the user will upload/export or even edit source data. On the left, you can see a list of uploaded csv's again. When the user selects one, they can also visualize the source data in a few simple charts or see the entire source data in a table. The cells of the table are modifiable so they can also edit the source data.

This tab uploads all sources directly to the database, using the backend services, while the csv file's are handled locally.

Our goal with each tab was achieving an easily navigable and understandable UI, while giving all the necessary tools for the plant-operators/users for adequate management. Although we only received source data for four weeks and two asset scenarios, we tried to create a scalable, modular UI, that can handle dozens of assets and years worth of source data. As a side goal we also wanted to achieve a simple yet modern and visually appealing look.

5.1.1 Midterm Version

For the midterm demo, we prioritised usability over aspect. As this version was a mere proof of concept, and its goal was to plant a picture of our way of implementing it and to show what we think is possible, some aspects of the overall user experience was ignored so we can focus on actual functionality.

This means that the UI for this version has a very generic, template look. Buttons are rough and placed where we could fit them, while icons are uninteresting and unintuitive. A majorly different approach for displaying assets, results and sources was used in this version. Instead of the frontend lists updating to changes accordingly (like uploading csv's or generating results) in real time, we added a refresh button that, when clicked, reloads all the data from the database. This solution was a compromise made for this version, and eliminating it, a focuspoint for the final version.

The Assets tab was fully functional, and only needed some visual and aesthetic improvements.

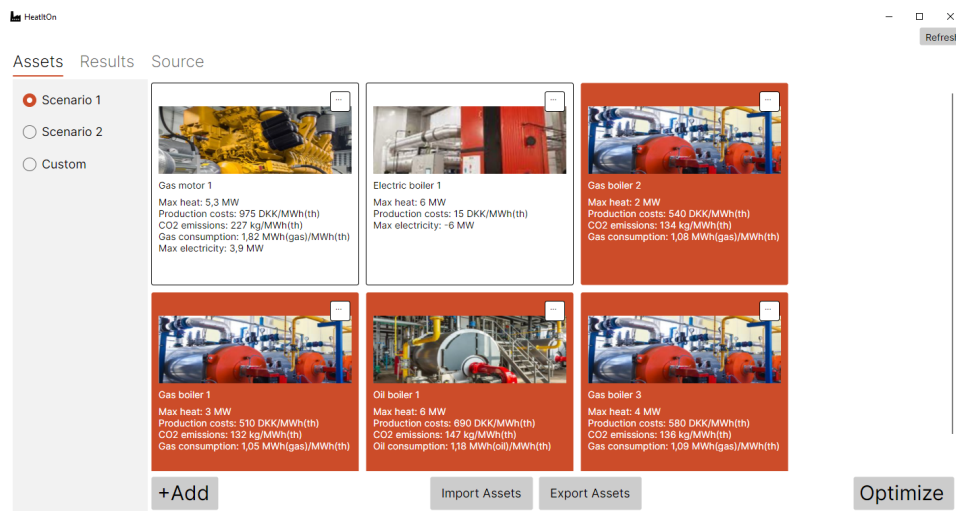


Figure 5.1: Assets tab from the demo version

The Results tab in this version is defined by minimalistic graphs, that made it hard to visualize the results. Our three graphs(electricity use/production, heat production and CO2 production) were a good start but we realised we need a lot more for proper visualization.

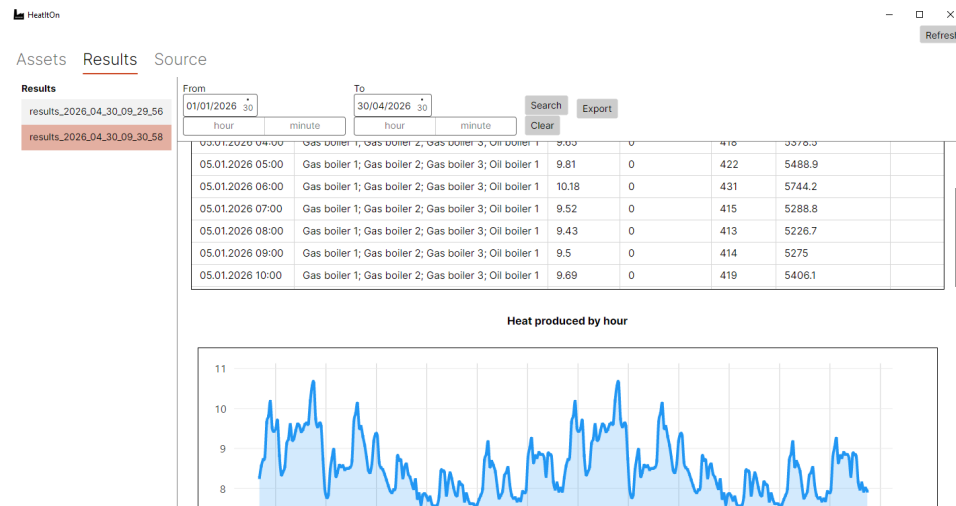


Figure 5.2: Results tab from the demo version

The Sources tab functioned as expected but the possibility to edit the csv data was missing from this version.

Overall the demo was a very solid proof of concept that helped us learn what we were missing for the final version while also showing us the direction we are going.

5.1.2 Final Version

The final version focuses more on user experience with minor functionality improvements when compared to the midterm implementation. The primary goal of this version was to improve usability, visual appearance, and introduce additional statistical aspects while maintaining a consistent and scalable structure.

A significant improvement is the change from a manual data refresh system to a dynamic, responsive interface. In contrast to the midterm version, where a dedicated refresh button was required to update data, the final implementation ensures that all lists and views update automatically in response to changes in the database. This enhancement improves workflow efficiency and eliminates unnecessary user interaction.

The visual design of the interface has been improved. The final version adopts a modern, structured design with consistent colors, clearer text formatting, and improved spacing. Asset cards have been redesigned to display key information in a meaningful and readable format, including maximum heat, production cost, CO₂ emissions, and electricity generation/consumption. Additionally, asset categories such as gas, oil, electric, and motor are visually distinguished through icons and labels, allowing for faster recognition and comparison.

The scenario selection system has been refined and better integrated into the workflow. Instead of simple toggles, the final version introduces clearly defined scenario panels with descriptive labels such as “Peak Load,” “Eco Mode,” and “Custom Configure.” Each scenario communicates its purpose more effectively, enabling more intuitive decision-making. Furthermore, visual indicators highlight selected assets within each scenario, improving clarity during configuration.

Additional controls have been added, including a “Manager” and an “Optimize” button. The Manager section serves as a centralized interface for creating, importing, and exporting assets. These improvements lead to a clearer workflow, guiding the user from setup to execution in a more structured way.

The Results tab has undergone major enhancements in both data presentation and analytical depth. A summary dashboard has been introduced at the top of the page, presenting key performance indicators such as total heat production, net electricity balance, total CO₂ emissions, and total production cost. These aggregated metrics provide immediate insight into optimization outcomes without requiring detailed inspection of raw data.

The data display has also been improved with better formatting, clearer column labeling, and pagination support for large datasets. The addition of pagination ensures that even large datasets remain readable and manageable.

The charting system represents one of the most significant functional im-

provements. While the midterm version included only basic visualizations, the final version introduces a comprehensive set of charts:

- A stacked area chart showing heat production per asset over time, including a total heat overlay
- A production cost per hour chart, allowing detailed financial analysis
- A generator share pie chart, illustrating the contribution of each asset within a selected time range
- An electricity usage and generation chart, displaying hourly electricity balance in MWh

These visualizations provide a deeper understanding of system behavior and allow users to analyze performance trends, cost distribution, and resource utilization more effectively. Interactivity elements, such as tooltips and time-range filtering, further enhance data exploration. The date and time filtering functionality has been refined, with a more structured input system for selecting precise time intervals. Filtering affects both tabular and graphical outputs, ensuring consistency across all displayed data.

The Sources tab has also been expanded with important new functionality. Unlike the midterm version, the final implementation allows direct editing of source data within the table. This feature enables on-the-fly adjustments without requiring external file modification. The table design has been improved for clarity, and pagination has been introduced to handle large datasets efficiently. File management capabilities have been enhanced, including clearer separation between uploaded files and active data views. Import and export functions remain available but are now better integrated into the interface.

Overall, the final version achieves a significantly higher level of usability, clarity, and analytical capability. The transition from a proof-of-concept interface to a polished, production-oriented design is evident across all tabs. The system now provides a comprehensive toolset for asset management, optimization execution, and result analysis, while maintaining a scalable and user-centered design approach.

5.2 Backend Implementation

5.2.1 Midterm Version

The backend is implemented using ASP.NET Core 9.0 and follows a layered architectural design that emphasizes modularity, scalability, and separation of concerns. The structure consists of Controllers, Services, Interfaces, Models, and a Data layer, each responsible for a distinct part of the system functionality.

Architecture and Main Components

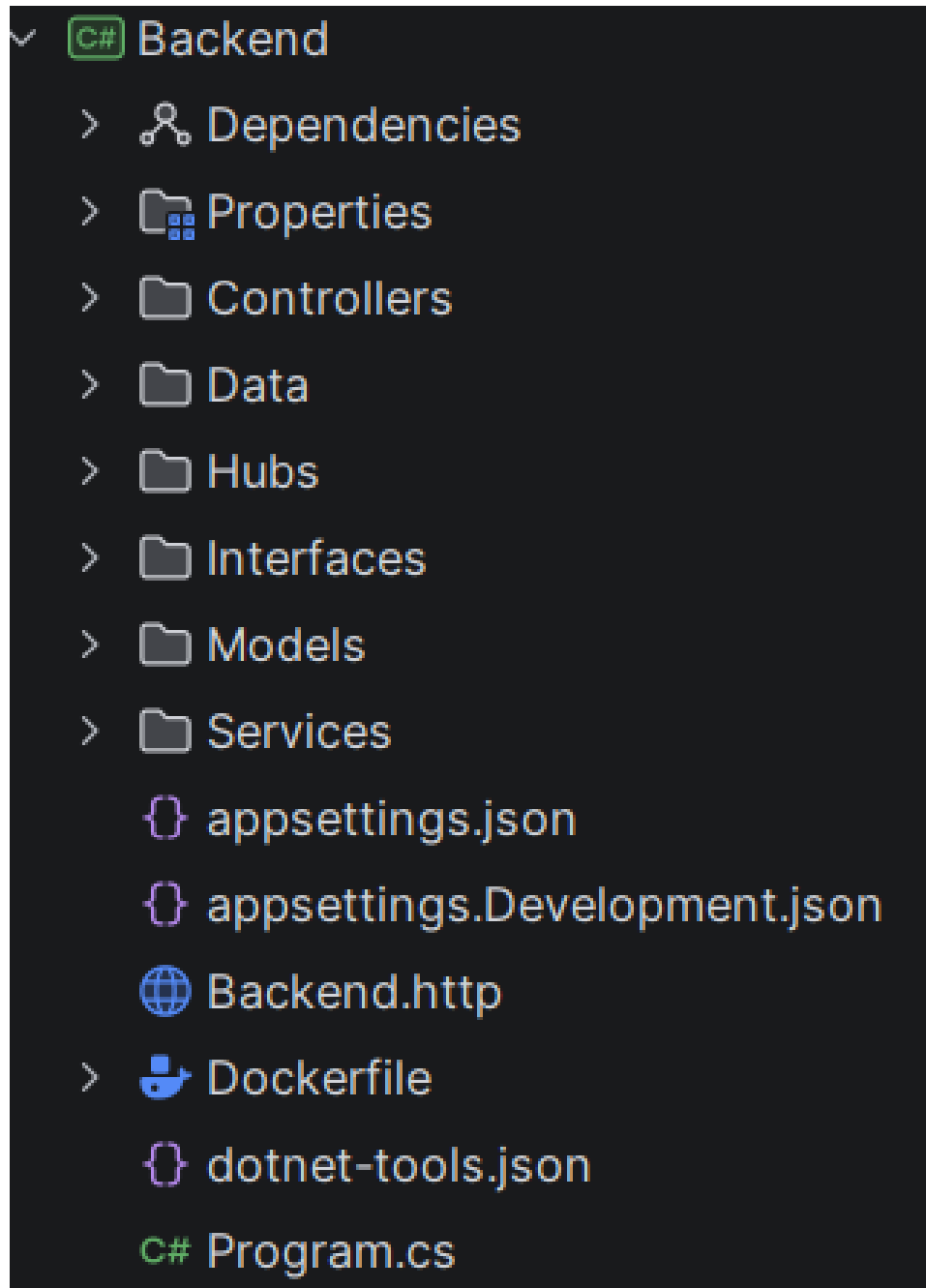


Figure 5.3: Backend file structure showing the organization of controllers, services, interfaces, models, and data components.

The backend is organized into clearly defined layers:

- **Controllers:** Provide RESTful API endpoints and act as the entry point for client requests. Controllers are responsible for handling HTTP requests, binding input parameters, and returning appropriate responses.
- **Services:** Implement the core business logic. Each service corresponds to a domain entity, such as assets, source data, and optimization results. Services coordinate data processing and communicate with the persistence layer.
- **Models:** Represent domain entities and map directly to database tables. Examples include `Asset`, `Source`, `Result`, `ResultList`, and `OptimizedResults`.
- **Data Layer:** Contains the Entity Framework database context (`BackendDbContext`) and migration files. This layer manages communication with the MySQL database.

This architecture ensures that responsibilities are clearly separated, enabling easier maintenance and extensibility.

API and Controller Design

During the midterm the backend exposed basic REST endpoints (CRUD) implemented ad-hoc, these were formalized in the final version through generic controller/service interfaces (see the **Final Version** subsection).

All controller actions are implemented asynchronously using `Task<IActionResult>`, allowing efficient handling of concurrent requests. Controllers delegate processing tasks to the service layer, keeping the API layer thin and focused on HTTP concerns.

Business Logic Handling

All business logic is implemented within the service layer, which acts as the core processing unit of the backend. Each service implements the generic `IService<T>` interface, providing consistent operations across all entities.

Specialized services, such as the `OptimizerService`, encapsulate domain-specific logic related to heat production optimization. This includes processing input data, calculating results, and coordinating interactions between assets, source data, and result data.

The separation of business logic from controllers improves maintainability and enables independent testing of core functionality.

Data Processing and Validation

Data processing is primarily handled within the service layer. Incoming data from API requests is validated before being persisted or processed further. Validation mechanisms include:

- Strongly typed C# models for structured data representation
- Parameter binding and validation features provided by ASP.NET Core

- Logical validation rules implemented within service methods

This approach ensures that invalid or inconsistent data is prevented from entering the system and maintains overall data integrity.

Database Communication

The backend communicates with the MySQL database using Entity Framework. The `BackendDbContext` manages entity tracking, query execution, and persistence.

- Models are mapped directly to database tables
- LINQ is used for querying and manipulating data
- Asynchronous database operations improve performance and scalability

Services interact with the database context to perform all data operations, ensuring controlled access to the persistence layer and preventing direct database access from controllers.

System Behavior Support

The backend supports the required system behavior by coordinating interactions between different system modules. Controllers expose endpoints for managing assets, source data, and optimization results, while services execute the corresponding logic.

The inclusion of a dedicated optimization service enables the processing of time-series data for heat demand and electricity prices, supporting the generation of optimized production schedules.

Deployment Considerations

The backend is deployed using Docker, which provides a consistent and isolated runtime environment. Containerization ensures that dependencies, configurations, and runtime settings are standardized across development and production environments.

This approach improves portability, simplifies deployment, and enhances the reliability of the backend infrastructure.

Overall, the backend implementation follows best practices for modern web application development, emphasizing modularity, scalability, and maintainability. The layered architecture ensures that the system can effectively support the required functionality while remaining adaptable to future enhancements.

5.2.2 Final Version

The final backend version extends the original layered ASP.NET Core architecture with additional runtime functionality and tighter system integration. We introduced two supporting components: a centralized exception-handling middleware and interface-based service contracts that define abstraction boundaries between controllers and services.

These additions work together with the existing layers as follows: controllers accept HTTP requests and delegate work to services; services implement business logic and interact with the data layer (EF `DbContext`); both controllers and services rely on the `Interfaces` abstraction and dependency injection; the `ExceptionHandlerMiddleware` sits early in the request pipeline, catching exceptions from controllers and services, logging them, mapping domain errors to HTTP status codes, and returning compact JSON error responses. As a result, many local try/catch blocks were removed and method implementations became easily readable.

- **Exception handling:** Centralized via a custom middleware (`ExceptionHandlerMiddleware`) that intercepts exceptions thrown by controllers and services, logs them, maps domain errors to appropriate HTTP status codes, and returns a compact JSON payload (for example `{ "error": "<message>", "stackTrace": "<stackTrace>" }`) with `Content-Type: application/json`. This provides consistent, machine-readable error responses for clients and prevents raw error details from leaking.
- **Interfaces:** Provide clear abstraction boundaries for controllers and services (e.g., `IController<TCreate, TUpdate>`, `IService<T>`). Using interfaces enables dependency injection, easier unit testing, and consistent error-typing so the middleware can map domain exceptions to appropriate HTTP responses.
 - The generic controller interface formalizes standard CRUD operations used across controllers:
 - * `List()` for retrieving collections of entities
 - * `Get(int id)` for retrieving a single entity
 - * `Post(TCreate model)` for creating new entries
 - * `Put(int id, TUpdate model)` for updating existing entries
 - * `Delete(int id)` for removing entries
- **Controllers:** Handle request/response concerns and remain thin — delegating business rules to services and relying on interfaces for testability, validate inputs.
- **Services:** Contain core business logic, interact with the data layer, and throw well-defined domain exceptions when needed. They do not implement HTTP concerns or repeated try/catch logic anymore.
- **Data layer:** The Entity Framework `DbContext` and related repositories/models remain responsible for persistence and are used by services through interface-driven abstractions.

5.3 Data Management

5.3.1 Midterm Version

Data management is implemented using a relational database approach supported by MySQL, combined with an object-relational mapping layer provided by Entity Framework.

Data Structure

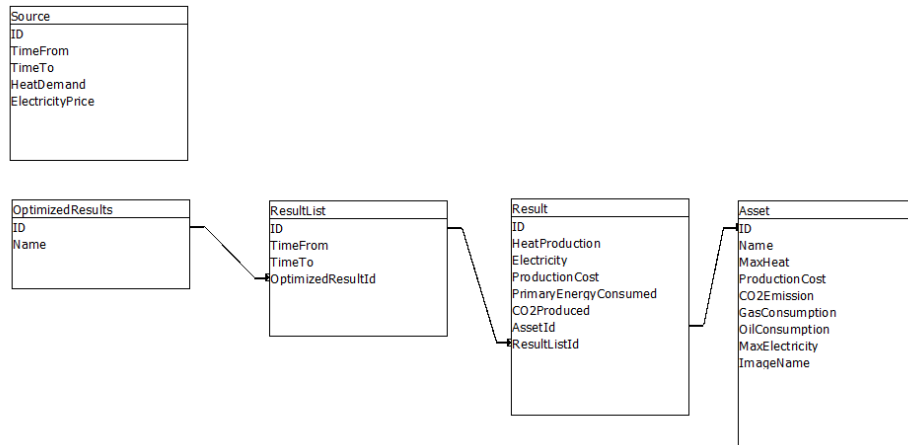


Figure 5.4: Database schema illustrating the relationships between Source, OptimizedResults, ResultList, Result, and Asset tables.

The data model is organized into five primary tables: Source, OptimizedResults, ResultList, Result, and Asset, each representing a distinct aspect of the system domain.

- The Source table stores time-based input data, including HeatDemand, and ElectricityPrice. This structure supports time-series data used for optimization processes.
- The OptimizedResults table represents a container for optimization runs during the selected time period.
- The ResultList table links individual time intervals to a specific optimization result through the foreign key OptimizedResultId. Each entry contains results for separate assets within an hourly time frame, with attributes such as TimeFrom and TimeTo defining the interval.
- The Result table contains detailed output data generated by the optimization process. Attributes include HeatProduction, Electricity, ProductionCost, PrimaryEnergyConsumed, and CO2Produced. This table also includes foreign keys AssetId and ResultListId, establishing

relationships with both Asset and ResultList. It stores the specific performance metrics for one asset in one hour.

- The Asset table defines the characteristics of generators or boilers, including MaxHeat, ProductionCost, CO2Emission, GasConsumption, OilConsumption, MaxElectricity, and ImageName.

The relationships between these tables follow a hierarchical structure:

- One OptimizedResults entity relates to multiple ResultList entries.
- One ResultList entry relates to multiple Result records.
- Each Result references a single Asset.

Primary keys and foreign keys ensure referential integrity across all tables. For example, Result.AssetId must correspond to a valid entry in the Asset table, and ResultList.OptimizedResultId must reference an existing optimization result.

Integration with Backend

The data structure integrates directly with the backend through Entity Framework. Each database table is mapped to a corresponding C# model, enabling seamless interaction between the application logic and the database layer. Entity Framework provides abstraction over raw SQL queries, allowing developers to interact with the database using LINQ queries and strongly typed objects. The DbContext component manages database connections, entity tracking, and query execution, ensuring consistent data access patterns across the application. Migration tools within Entity Framework support schema evolution, allowing controlled updates to the database structure while maintaining data integrity.

Data Management Environment

The database is deployed using Docker, ensuring a consistent and reproducible environment. The use of Docker also facilitates easy deployment, testing, and version control of the data management infrastructure.

Overall, the data management approach combines a well-defined relational schema, validation mechanisms, and seamless backend integration to ensure reliability, scalability, and maintainability of the system.

5.3.2 Final Version

The recent database updates were relatively minor and focused on strengthening data integrity. We refined several model-level constraints using data-annotation attributes (e.g., Required, Range, MaxLength) to ensure that invalid or inconsistent values cannot be stored. Additionally, a unique constraint was introduced for the Source entity through the DbContext configuration, preventing duplicate source entries from being created. These

adjustments improve validation and consistency across the system without modifying the core schema.

Chapter 6

Discussion and Reflection

This chapter discusses the software engineering practices and processes adopted throughout the project. It also contains potential future work that could further improve the product.

6.1 Scrum

“Scrum is a lightweight framework that helps people, teams and organizations generate value through adaptive solutions for complex problems.”[?] Reflecting on the team’s use of Scrum, the framework had a positive impact on the team’s workflow and collaboration. Compared to the previous semester project, where the work was less structured, Scrum introduced a level of organization that proved genuinely useful to the team.

Sprint Planning helped the team stay aligned by dividing tasks early and estimating effort through story points, which made it easier to identify when a sprint was becoming overloaded. Daily Scrums, held two times a week, improved our cooperation. Team members were more aware of each other’s progress and blockers, which reduced situations where work was duplicated or misunderstood.

The Sprint Retrospective proved to be helpful for the team. Using the Start Doing, Stop Doing, and Keep Doing tables in Jira, the team was able to identify and discuss issues such as reviews being rushed, tasks lacking clear descriptions, and errors that made it past the review stage. Based on this specific changes were agreed on for the following sprint. This led to gradual but visible improvements in how the team cooperated over time.

Overall, Scrum contributed to better organisation, more open communication throughout the project.

6.2 Requirements Engineering

“Requirements engineering is the process of understanding and defining what services are required from the system and identifying the constraints on the system’s operation and development.” [?] Practises used during this project:

- User stories - Helped the team understand the user perspective and ensure the application was developed with real user needs in mind, contributing to a more user-friendly product
- Product Backlog - Provided a full and organised list of all tasks in one place, making it easier to manage and prioritise work across the project
- Functional requirements - Ensured all members of team understood what the software is supposed to do
- Non-functional requirements - Ensured all members of team understood the quality standard the software needed to meet
- Story points - Estimated the effort for all the tasks
- Sprint backlog - Helped the team stay focused on tasks chosen from the backlog for the current sprint
- Reviews with the team - Allowed each member to validate certain tasks helping to reveal and prevent mistakes and errors
- Definition of Done - Determined what finished task mean (code, testing, reviews)

In the future, all of these practices will be used again. User stories will be written in more detail, the product backlog will be prepared more thoroughly at the beginning of the project, and reviews will be taken more seriously.

6.3 Refactoring

Throughout the project, several refactoring improvements were made to keep the codebase clean and maintainable. CSV-related functionality was combined into a single unified class to reduce duplication. A centralized error handler was introduced, removing repetitive try-catch blocks from services and controllers. Data annotations in the models were expanded for better input validation, and interfaces were added to improve code structure. Unused imports were removed, formatting was cleaned up, and inheritance structures were reviewed and adjusted where needed.

6.4 Code review

During this project, two reviewers were assigned to every task in Jira and GitHub. Both had to either approve or request changes, and all issues had to be resolved before merging the work into the main branch. This practice helped improve overall code quality and catch bugs early in the process.

However, this practice came with some challenges. Early in the project, some reviews were delayed or not done properly, which occasionally led to mistakes slipping through. To address this, the team introduced a deadline system midway through the project. All tasks had to be completed by Wednesday of the second sprint week, and all reviews had to be finished by the following Thursday. This was not only to prevent reviews from being rushed at the last minute, but also to ensure Fridays could be dedicated fully to the Sprint Review and Retrospective, rather than still catching up on unfinished work.

6.5 Testing

During the project, unit tests were written for every task that involved business logic. Testing was part of the Definition of Done, meaning tasks could not go for review until the necessary tests were written.

Manual end-to-end testing was conducted to ensure the application ran and behaved as expected. For tasks involving the database, Postman was used to test every endpoint after new code was added or modified.

Unit testing proved to be one of the more challenging parts of the project, as it was a relatively new practice for most team members. Early on, tests were occasionally forgotten and written retrospectively, or not done properly due to inexperience. Over time, however, the team became more comfortable with writing tests along with their implementation. Unit tests played an important role in catching bugs early in the code and verifying that each part of the code worked as expected on its own. In future projects, the team will try to adopt a habit of writing tests from the start, and invest more time in reviewing test quality alongside code quality.

Chapter 7

Conclusion

7.1 Conclusion

HeatItOn is a utility in the city of Heatington, that supplies heat to approximately 1600 buildings through a district heating network. Before this project, heat production was planned manually, making it difficult to always select the most efficient combination of production units. The goal was to replace this manual process with an automated optimization system that was capable of generating heat production schedules that meet demand at all times while minimizing operational costs.

The system was built around five core modules. The Optimizer, Data Visualization, Asset Manager, Source Data Manager, and Result Data Manager, each responsible for a specific role within the application. The architecture followed a clear separation between Frontend, Backend, and Database. Frontend was built using AvaloniaUI following the MVVM pattern, the Backend implemented in ASP.NET Core 9.0 and the Database running MySQL through Entity Framework. Design principles such as SOLID, DRY, and Separation of Concerns were applied throughout the project, ensuring the code remained clear and maintainable.

Both required scenarios were successfully implemented.

Scenario 1 covered a heat only network with three gas boilers and one oil boiler, where the system prioritized cheaper units and reserved the oil boiler for peak demand periods. Maintenance scheduling was also handled, with one gas boiler taken offline for a window of 30 to 60 hours.

Scenario 2 introduced electricity production, working with two gas boilers, a gas motor, and an electric boiler. The optimizer calculated net production costs on an hourly basis by comparing each unit's production cost against the current electricity price, choosing to sell electricity when prices were high and consume it when prices were low.

In addition, a custom scenario was implemented, giving users control over which assets are included in the optimization.

The final product successfully addressed the original problem statement. Team managed to deliver an automated optimization system that replaces manual planning. Looking ahead, the system could be further extended by integrating live electricity prices from Energinet.dk, enabling or disabling units directly from the UI.

Chapter 8

References

8.1 References

This is our References.

Chapter 9

AI Usage

Throughout the project, the team incorporated AI across multiple stages to assist with various tasks. Importantly, AI was never used as a replacement for human effort or core decision-making. Instead, it served exclusively as a supplementary tool to streamline team workflows and optimize overall time management.

Team had established a strict internal policy regarding AI assistance:

- **Sandboxed Interaction:** It was decided that AI models should be intentionally kept isolated from the development environment. Direct integration tools (such as IDE-embedded copilots that autonomously suggest or alter code) were prohibited. This ensured that no AI-generated code could interact directly with the active codebase.
- **Manual Verification and Rewriting:** Any logic, syntax, or formatting suggested by AI in external chats was carefully analyzed. No code was ever pasted directly into the project without proper human verification, testing, and manual rewriting to fit the specific architecture.
- **Total Accountability:** The team takes full responsibility for all final code, design choices, and documentation. AI was treated purely as an external advisory tool. Therefore, the team assumes complete ownership of the final implementation as if it were 100% manually produced, including any flaws or mistakes.
- **Original Authorship:** While AI assisted with debugging LaTeX compilation errors and generating UML diagrams, all written content, structural logic, and academic analysis within this report are entirely the original work of the team members. No part of the report text was authored by AI.

Some of the examples using AI in this project:

- **Architectural Brainstorming:** In the early planning stages, AI was used to help generate ideas for potential design patterns and library integrations. The team then carefully weighed these options against established non-functional requirements before finalizing the architecture.
- **Debugging Assistance:** Dealing with undocumented Avalonia UI errors or sudden backend disconnects sometimes meant decoding vague error codes. AI was utilized as a quick-reference resource to parse these codes, though diagnosing the root cause within our specific architecture and developing the actual fixes remained a manual team effort.
- **Formatting and Documentation:** To streamline team workflow, it was decided to outsource repetitive formatting tasks to AI tools. This included generating sketches and resolving some of tricky LaTeX compilation errors in the report, which saved valuable time without affecting the design or logic of the core system.
- **UI/UX Inspiration:** Various stylistic directions were explored for project logos and interface by generating mood boards with AI image tools. Even though the final application relies exclusively on original assets, these early visualizations helped steer overall design choices.
- **Code Review and Refactoring:** During the refactoring phase, AI acted as an extra set of eyes to catch missed references. When renaming methods within optimization modules, for instance, it helped verify that all old instances were properly updated, effectively reducing minor human errors without altering the underlying code logic.

AI models that were used by team members:

- | | |
|----------------------|--------------------|
| • Gemini-3.1 Pro | • DeepSeek V3.2/V4 |
| • Claude Opus/Sonnet | • ChatGPT |
| • Microsoft Copilot | • GPT-5.3 Codex |

Appendix A

Appendix

A.1 Appendices



Figure A.1: Full frontend and backend detailed UML class diagram.